

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático 3: Jogo da Vida

BCC202 - Estruturas de Dados 1

Fábio Tavares Pinto e Hugo Augusto Silva de Faria
Professor: Pedro Henrique Lopes Silva

Ouro Preto
9 de outubro de 2024

Sumário

1	Introdução	1
2	Arquivo de Cabeçalho (double hash.h)	1
2.1	Código	1
2.2	Estrutura 'HashEntry'	2
2.3	Estrutura 'HashTable'	2
2.4	Funções Declaradas	2
3	Arquivo de Implementação (double hash.c)	3
3.1	Função Hash1	3
3.2	Função Hash2	3
3.3	Função inicializarHash	3
3.4	Função inserirHash	4
3.5	Função buscarHash	4
3.6	Função desalocarHash	5
4	Arquivo de Cabeçalho (automato.h)	5
4.1	Código	5
4.2	Estrutura 'Automato'	6
4.3	Funções Declaradas	6
5	Arquivo de Implementação (automato.c)	6
5.1	Função alocarReticulado	6
5.2	Função desalocarReticulado	7
5.3	Função LeituraReticulado	7
5.4	Função evoluirReticulado	8
5.5	Função imprimeReticulado	9
6	Arquivo de Implementação (tp.c)	10
6.1	Inclusão de Cabeçalhos	10
6.2	Função main	10
6.3	Chamadas das Funções	11
6.4	Fim da Função main	11
7	Impressões gerais	11
8	Análise	12
8.1	Tempo de Execução	12
8.2	Valgrind	12
8.3	Comparação em relação aos trabalhos anteriores	12
9	Conclusão	12

1 Introdução

O Jogo da Vida, proposto por John Conway em 1970, é um autômato celular que exemplifica como regras simples podem levar à emergência de comportamentos complexos. Este jogo, amplamente estudado em ciência da computação e matemática, é jogado em uma grade bidimensional onde cada célula pode estar viva ou morta. A evolução do sistema ocorre em iterações, e o estado futuro de cada célula é determinado pelo número de vizinhos vivos, seguindo um conjunto de regras fixas.

Neste trabalho, foi desenvolvido um simulador do Jogo da Vida utilizando a linguagem C. Para otimizar a manipulação das células vivas e mortas, foi implementada uma tabela hash com endereçamento duplo, que armazena o estado do tabuleiro de forma eficiente. O objetivo principal é garantir a correta evolução do autômato ao longo de várias gerações, mantendo a eficiência em termos de tempo e uso de memória.

As principais funcionalidades deste simulador incluem:

- Alocação e desalocação de memória: Inicialização eficiente da tabela hash para o armazenamento das células e liberação da memória após o uso, evitando vazamentos.
- Leitura e impressão do tabuleiro: Funções que leem o estado inicial das células e imprimem o estado do tabuleiro ao final de cada iteração, permitindo a visualização do progresso do jogo.
- Evolução do sistema: Implementação das regras do Jogo da Vida, onde o estado de cada célula é atualizado com base na contagem de seus vizinhos vivos, permitindo a simulação das gerações.

Este relatório descreve a estrutura do simulador, detalha as funções implementadas e discute o comportamento observado. Através deste projeto, investigamos como padrões emergentes podem surgir de sistemas governados por regras simples e exploramos o uso de autômatos celulares em simulações computacionais.

2 Arquivo de Cabeçalho (double hash.h)

O arquivo de cabeçalho hash.h define a interface para o módulo que implementa a manipulação de uma tabela hash com endereçamento aberto. Ele contém as declarações de tipos de dados e funções utilizadas para inicializar, inserir, buscar, remover e desalocar a tabela hash. A seguir, estão descritos os principais componentes e suas funções:

2.1 Código

```
1 #pragma once
2
3 // Estrutura da Tabela Hash
4 typedef struct {
5     int key; // A chave que representa uma celula viva (1) ou morta (0)
6     int value; // Valor associado a chave, no caso do Jogo da Vida, sera 1 (
7         vivo) ou 0 (morto)
8     int is_occupied; // 0 se a posicao esta vazia, 1 se esta ocupada
9 } HashEntry;
10
11 // Estrutura principal da Tabela Hash com endereçamento aberto
12 typedef struct {
13     int size; // Tamanho da tabela
14     HashEntry *table; // Array que representa a tabela hash
15     int *pesos; // Vetor de pesos para as funcoes hash
16 } HashTable;
17
18 // Funcao hash1
19 int hash1(int key, int size, int *pesos);
20
21 // Funcao hash2
22 int hash2(int key, int size, int *pesos);
```

```

22
23 // Funcao para inicializar a Tabela Hash
24 HashTable *inicializarHash(int size);
25
26 // Funcao para inserir um elemento na Tabela Hash
27 void inserirHash(HashTable *hashTable, int key, int value);
28
29 // Funcao para buscar um elemento na Tabela Hash
30 int buscarHash(HashTable *hashTable, int key);
31
32 // Funcao para desalocar a Tabela Hash
33 void desalocarHash(HashTable **hashTable);

```

2.2 Estrutura 'HashEntry'

Essa parte do código define uma estrutura chamada HashEntry, que é usada para representar uma entrada na tabela hash. A estrutura possui três campos::

- **int key:** Armazena a chave que representa uma célula viva (1) ou morta (0).
- **int value:** Armazena o valor associado à chave, que no caso do Jogo da Vida será 1 (vivo) ou 0 (morto).
- **int is_occupied:** Indica se a posição na tabela está ocupada (1) ou vazia (0).

2.3 Estrutura 'HashTable'

A estrutura HashTable : é definida para representar a tabela hash. Ela contém:

- **int size:** Um inteiro que representa o tamanho da tabela.
- **HashEntry *table:** Um ponteiro para um array de HashEntry, que representa a tabela hash.
- **int *pesos:** Um vetor de pesos utilizado para as funções hash.

2.4 Funções Declaradas

- ***int hash1(int key, int size, int pesos):** Esta função implementa a primeira função hash. Ela calcula um índice na tabela hash baseado na chave fornecida, utilizando os pesos especificados.
- ***int hash2(int key, int size, int *pesos):** Esta função implementa a segunda função hash, semelhante à primeira, mas com uma lógica de hash diferente, permitindo a resolução de colisões.
- ***HashTable inicializarHash(int size):** Esta função aloca memória para uma nova tabela hash e inicializa suas entradas, retornando um ponteiro para a estrutura HashTable recém-alocada.
- ***void inserirHash(HashTable hashTable, int key, int value):** Esta função insere um novo elemento na tabela hash, utilizando as funções hash para encontrar um índice apropriado e tratando colisões conforme necessário.
- ***int buscarHash(HashTable hashTable, int key):** Busca e retorna o valor associado à chave fornecida na tabela hash. Se a chave não existir, a função retorna um valor que indica a ausência.
- ****void desalocarHash(HashTable hashTable):** Libera a memória alocada para a tabela hash, garantindo que todos os recursos sejam corretamente desalocados.

3 Arquivo de Implementação (double hash.c)

O arquivo de implementação double.hash.c contém a definição das funções declaradas no arquivo de cabeçalho double.hash.h. Essas funções são responsáveis pela inicialização, inserção, busca, remoção e desalocação de elementos em uma tabela hash que utiliza o método de hashing duplo. A seguir, estão descritas as principais funções e sua implementação:

3.1 Função Hash1

```
1 int hash1(int key, int size, int *pesos) {
2     return (key * pesos[0] + pesos[1]) % size;
3 }
```

A função hash1 é a principal função de hash que gera o índice inicial para inserir ou buscar um elemento na tabela hash. Ela usa uma chave (key), o tamanho da tabela (size), e um vetor de pesos para gerar um valor único. A fórmula básica é $(key * pesos[0] + pesos[1]) \% size$, onde os pesos adicionam variação ao cálculo, ajudando a distribuir melhor as chaves na tabela, e o operador módulo (%) garante que o índice esteja dentro dos limites da tabela (de 0 até size-1).

3.2 Função Hash2

```
1 int hash2(int key, int size, int *pesos) {
2     // Para evitar o hash2 de ser zero, usamos (pesos[2] + key) % (size - 1) +
      1
3     return (key * pesos[2] + pesos[3]) % (size - 1) + 1;
4 }
```

A função hash2 é usada para resolver colisões. Ela gera um "passo" (offset) diferente para cada chave, calculado pela fórmula $(key * pesos[2] + pesos[3]) \% (size - 1) + 1$. Esse "passo" garante que, em caso de colisão, o elemento será realocado em outro índice, permitindo um duplo hashing.

3.3 Função inicializarHash

```
1 // Inicializacao da Tabela Hash
2 HashTable *inicializarHash(int size) {
3     HashTable *hashTable = (HashTable *)malloc(sizeof(HashTable));
4     if (hashTable == NULL) {
5         printf("Erro de memoria");
6     }
7     return NULL;
8
9     hashTable->size = size;
10    hashTable->table = (HashEntry *)malloc(size * sizeof(HashEntry));
11    if (hashTable->table == NULL) {
12        printf("Erro de memoria");
13        free(hashTable);
14    }
15    return NULL;
16
17    // Inicializa as entradas como nao ocupadas
18    for (int i = 0; i < size; i++) {
19        hashTable->table[i].is_occupied = 0;
20    }
21
22    // Aloca e inicializa o vetor de pesos
23    hashTable->pesos = (int *)malloc(4 * sizeof(int));
24    hashTable->pesos[0] = 1;
25    hashTable->pesos[1] = 2;
26    hashTable->pesos[2] = 3;
27    hashTable->pesos[3] = 4;
28    return hashTable;
29 }
```

28 }

A função `inicializarHash` aloca memória para uma tabela hash de um determinado tamanho. Ela cria a estrutura `HashTable`, inicializa a tabela de entradas como não ocupadas e aloca um vetor de pesos para as funções de hash. Se alguma alocação falhar, uma mensagem de erro é impressa e a função retorna `NULL`; caso contrário, retorna um ponteiro para a tabela hash criada. Sua complexidade é de $O(m)$ porque o tempo de execução da função depende linearmente do tamanho da tabela hash, m é a dimensão da matriz.

3.4 Função `inserirHash`

```
1 // Insercao na Tabela Hash utilizando dupla hashing
2 void inserirHash(HashTable *hashTable, int key, int value) {
3     int index = hash1(key, hashTable->size, hashTable->pesos);
4     int step = hash2(key, hashTable->size, hashTable->pesos);
5     while (hashTable->table[index].is_occupied) {
6         if (hashTable->table[index].key == key) {
7             // Atualiza o valor se a chave ja existe
8             hashTable->table[index].value = value;
9             return;
10        }
11    // Avanca para o proximo indice utilizando a funcao de hashing duplo
12    index = (index + step) \% hashTable->size;
13 }
14
15 // Insere a nova entrada
16 hashTable->table[index].key = key;
17 hashTable->table[index].value = value;
18 hashTable->table[index].is_occupied = 1;
19 }
```

A função `inserirHash` insere uma nova entrada na tabela hash utilizando o método de hashing duplo. Ela calcula o índice inicial usando `hash1` e o passo de deslocamento usando `hash2`. Se a posição já estiver ocupada e a chave já existir, o valor é atualizado; caso contrário, a nova entrada é inserida na posição apropriada. A complexidade dessa função depende do número de colisões que ocorrem durante a inserção, no pior caso a complexidade é $O(m)$ devido à possibilidade de percorrer até m entradas durante o processo de resolução de colisões.

3.5 Função `buscarHash`

```
1 // Busca na Tabela Hash utilizando dupla hashing
2 int buscarHash(HashTable *hashTable, int key) {
3     int index = hash1(key, hashTable->size, hashTable->pesos);
4     int step = hash2(key, hashTable->size, hashTable->pesos);
5     while (hashTable->table[index].is_occupied) {
6         if (hashTable->table[index].key == key) {
7             return hashTable->table[index].value;
8         }
9         // Avanca para o proximo indice utilizando a funcao de hashing duplo
10        index = (index + step) \% hashTable->size;
11    }
12    return -1; // Indica que a chave nao foi encontrada
13 }
```

A função `buscarHash` procura por uma chave específica na tabela hash. Ela utiliza a função de hashing duplo para encontrar o índice inicial e, em seguida, percorre a tabela até encontrar a chave ou determinar que ela não existe, retornando `-1` nesse último caso. A complexidade dessa função também depende de quantas colisões ocorreram na tabela hash, o pior caso é $O(m)$ e o melhor caso é se não houver colisões e a chave for encontrada imediatamente, que seria $O(1)$.

3.6 Função desalocarHash

```
1 // Desalocacao da Tabela Hash
2 void desalocarHash(HashTable **hashTable) {
3     if (hashTable == NULL || *hashTable == NULL) {
4         return;
5     }
6
7     free((*hashTable)->table);
8     free((*hashTable)->pesos); // Libera o vetor de pesos
9     free(*hashTable);
10    *hashTable = NULL;
11 }
```

A função `desalocarHash` libera a memória ocupada pela tabela hash e seu vetor de pesos. Ela verifica se o ponteiro da tabela hash é válido antes de proceder com a desalocação, definindo o ponteiro como `NULL` para evitar acessos a memória desalocada. A complexidade dessa função é $O(1)$, visto que a função realiza operações independentes do tamanho da tabela.

4 Arquivo de Cabeçalho (automato.h)

O arquivo de cabeçalho `automato.h` define a interface para o módulo que implementa a evolução de um autômato celular, representado por uma tabela hash com endereçamento duplo. Ele contém as declarações de funções utilizadas para inicializar o autômato, ler o estado inicial, evolui-lo ao longo de várias gerações e imprimir o estado atual na tela. A seguir, estão descritos os principais componentes e suas funções:

4.1 Código

```
1 #pragma once
2
3 #include "double_hash.h"
4
5 // Definicao do tipo Automato
6 typedef struct
7 {
8     int n; // Dimensao do Automato (n x n)
9     HashTable *tabela; // Ponteiro para a tabela hash que representa o
10     Automato
11 } Automato;
12
13 // Funcao para inicializar o automato
14 Automato *alocarReticulado(int n);
15
16 // Funcao para desalocar a memoria do Automato
17 void desalocarReticulado(Automato **tab);
18
19 // Funcao para ler o Automato
20 void LeituraReticulado(Automato *tab);
21
22 // Funcao para evoluir o estado do Automato para a geracao N de acordo com as
23 regras do Jogo da Vida
24 Automato *evoluirReticulado(Automato *tab, int n, int geracoes);
25
26 // Funcao para imprimir o estado do Automato na tela
27 void imprimeReticulado(Automato *tab);
```

4.2 Estrutura 'Automato'

Essa parte do código define uma estrutura chamada **Automato**, que é utilizada para representar um autômato celular, que evolui ao longo de várias gerações em um reticulado. A estrutura possui dois campos:

- **int n**: Um inteiro que representa a dimensão do reticulado do autômato ($n \times n$). Ele indica o tamanho da "tabela" onde o autômato evolui.
- ***HashTable tabela**: Um ponteiro para uma tabela hash que representa o estado atual do autômato. Essa tabela é usada para armazenar as células vivas do reticulado em um formato eficiente de busca, evitando o uso de uma matriz esparsa tradicional e permitindo simular o Jogo da Vida de maneira otimizada.

4.3 Funções Declaradas

- **Automato *alocarReticulado(int n)**: Esta função é responsável por alocar memória para um novo autômato, baseado em um reticulado de dimensão $n \times n$. Ela cria e inicializa a tabela hash associada ao autômato e retorna um ponteiro para a estrutura recém-allocada. Se a alocação falhar, a função retornará NULL. Assim, essa função prepara o ambiente para a evolução do autômato.
- **void desalocarReticulado(Automato **tab)**: Essa função libera a memória alocada para o autômato, evitando vazamentos de memória. Ela desaloca primeiramente a tabela hash associada ao autômato e, em seguida, libera a memória da própria estrutura Automato. Ao passar um ponteiro para um ponteiro, ela permite a liberação completa do recurso de memória.
- **void LeituraReticulado(Automato *tab)**: Essa função lê o estado inicial do autômato a partir da entrada padrão. A leitura é utilizada para configurar as células vivas no reticulado, ou seja, na tabela hash, definindo o estado inicial do autômato para a simulação do Jogo da Vida. A função popula a tabela hash com as informações fornecidas pelo arquivo de entrada.
- **Automato *evoluirReticulado(Automato *tab, int n, int geracoes)**: Essa função aplica as regras de evolução do Jogo da Vida ao autômato, ao longo de um número especificado de gerações. A cada geração, o estado do reticulado na tabela hash é atualizado, conforme as regras do jogo, modificando as posições das células vivas. O retorno da função é o autômato atualizado, pronto para evoluir para a próxima geração ou ser impresso.
- ***void imprimeReticulado(Automato *tab)**: Esta função imprime o estado atual do reticulado do autômato na tela, permitindo a visualização do autômato após cada evolução. Ela percorre a tabela hash e exibe as células vivas, mostrando a configuração atual do reticulado.

5 Arquivo de Implementação (automato.c)

O arquivo de implementação **automato.c** contém a definição das funções declaradas no arquivo de cabeçalho **automato.h**. Essas funções são responsáveis pela alocação, leitura, evolução e impressão do estado de uma matriz representada por uma tabela hash no contexto do Jogo da Vida. A seguir, estão descritas as principais funções e sua implementação:

5.1 Função **alocarReticulado**

```
1 Automato *alocarReticulado(int n)
2 {
3     Automato *tab = (Automato *)malloc(sizeof(Automato));
4     if (tab == NULL)
5     {
6         printf("Erro de memoria\\textbackslash{n}");
7         return NULL;
8     }
```



```

9     tab->n = n;
10    tab->tabela = inicializarHash(n * n); // Inicializa a Tabela Hash com
        tamanho adequado
11    if (tab->tabela == NULL)
12    {
13        printf("Erro de memoria ao inicializar a tabela hash\\textbackslash{}n"
14              );
15        free(tab);
16        return NULL;
17    }
18    return tab;

```

A função `alocarReticulado` é responsável por criar e inicializar uma nova instância da estrutura `Automato`. Ela recebe um inteiro `n`, que define a dimensão da matriz a ser representada pela tabela hash. A função começa alocando memória para a estrutura `Automato` e, em seguida, inicializa a tabela hash com o tamanho adequado. Se qualquer etapa falhar, a função desaloca a memória e retorna `NULL`. Essa função tem complexidade $O(m^2)$, pois a operação dominante na função é a inicialização da Tabela Hash com m^2 entradas.

5.2 Função desalocarReticulado

```

1 void desalocarReticulado(Automato **tab)
2 {
3     if (tab == NULL || *tab == NULL || (*tab)->tabela == NULL)
4     {
5         return;
6     }
7
8     desalocarHash(&((*tab)->tabela)); // Desaloca a Tabela Hash
9     free(*tab);
10    *tab = NULL;
11 }

```

A função `desalocarReticulado` libera a memória associada ao objeto `Automato` e à tabela hash. Ela verifica se os ponteiros são válidos antes de liberar a memória. A tabela hash é desalocada usando a função `desalocarHash`, e o ponteiro para a estrutura é definido como `NULL` para evitar acessos inválidos. A operação dominante nesta função é a desalocação da Tabela Hash, que possui tamanho m^2 . Portanto, a complexidade total da função é $O(m^2)$.

5.3 Função LeituraReticulado

```

1 void LeituraReticulado(Automato *tab)
2 {
3     int valor;
4     for (int i = 0; i < tab->n; i++)
5     {
6         for (int j = 0; j < tab->n; j++)
7         {
8             scanf("%d", &valor);
9             if (valor == 1)
10             { // Se a celula esta viva, insere na Tabela Hash
11                 inserirHash(tab->tabela, i * tab->n + j, 1);
12             }
13         }
14     }
15 }

```

A função `leituraReticulado` é responsável por ler o estado inicial da matriz representada na tabela hash. A função percorre cada célula da matriz e, para cada célula viva (representada por 1), insere a posição correspondente na tabela hash. Isso permite representar eficientemente células vivas

em uma estrutura otimizada. A complexidade da função é $O(m^2)$ devido aos dois laços aninhados que percorrem todos os elementos de uma matriz $m \times m$.

5.4 Função evoluirReticulado

```
1 Automato *evoluirReticulado(Automato *tab, int n, int geracoes)
2 {
3     if (geracoes == 0)
4     {
5         return tab;
6     }
7
8     Automato *novoAutomato = alocarReticulado(n);
9     if (novoAutomato == NULL)
10    {
11        return NULL;
12    }
13
14    for (int i = 0; i < n; i++)
15    {
16        for (int j = 0; j < n; j++)
17        {
18            int key = i * n + j;
19            int vizinhosVivos = 0;
20
21            // Contagem de vizinhos vivos
22            for (int di = -1; di <= 1; di++)
23            {
24                for (int dj = -1; dj <= 1; dj++)
25                {
26                    if (di == 0 && dj == 0)
27                        continue;
28
29                    int ni = i + di;
30                    int nj = j + dj;
31                    if (ni >= 0 && ni < n && nj >= 0 && nj < n)
32                    {
33                        int vizinhoKey = ni * n + nj;
34                        if (buscarHash(tab->tabela, vizinhoKey) == 1)
35                        {
36                            vizinhosVivos++;
37                        }
38                    }
39                }
40            }
41
42            int cellState = buscarHash(tab->tabela, key);
43            if (cellState == 1)
44            { // Se a célula esta viva
45                if (vizinhosVivos == 2 || vizinhosVivos == 3)
46                {
47                    inserirHash(novoAutomato->tabela, key, 1); // Permanece
48                        viva
49                }
50            }
51            else
52            { // Se a célula esta morta
53                if (vizinhosVivos == 3)
54                {
55                    inserirHash(novoAutomato->tabela, key, 1); // Torna-se
56                        viva
57                }
58            }
59        }
60    }
61    return novoAutomato;
62 }
```

```

55     }
56 }
57 }
58 }
59
60     desalocarReticulado(&tab); // Desaloca o automato anterior
61
62     return evoluirReticulado(novoAutomato, n, geracoes - 1);
63 }

```

A função `evoluirReticulado` é responsável por evoluir o estado de um autômato celular que segue as regras do Jogo da Vida de Conway. O objetivo dessa função é calcular o estado do autômato após a aplicação das regras de evolução a partir do estado atual. A função recebe como parâmetro um ponteiro para a estrutura `Automato`, que contém a tabela hash representando as células vivas.

O processo começa percorrendo a tabela hash, onde estão armazenadas as células vivas. Para cada célula viva, a função conta o número de vizinhos vivos ao redor, verificando as oito direções ao redor da célula, exceto a própria posição da célula.

Com base no número de vizinhos vivos, as regras do Jogo da Vida são aplicadas para determinar o estado da célula na próxima geração:

- Se a célula está viva e possui 2 ou 3 vizinhos vivos, ela permanece viva.
- Se a célula está viva e tem menos de 2 ou mais de 3 vizinhos vivos, ela morre (por subpopulação ou superpopulação).
- Se a célula está morta e tem exatamente 3 vizinhos vivos, uma nova célula nasce nessa posição.

Além disso, a função verifica as células vizinhas que estão mortas para avaliar se elas devem se tornar vivas, aplicando a regra de nascimento. A tabela hash é atualizada conforme essas regras são aplicadas.

Após a atualização de todas as células, o estado final da tabela hash é atualizado, representando o novo estado do reticulado após a evolução. Esse processo pode ser repetido por várias gerações, permitindo a evolução contínua do autômato ao longo do tempo.

A função `evoluirReticulado` tem uma complexidade de $O(n^2 \cdot g)$, onde n é o tamanho do tabuleiro e g é o número de gerações que queremos simular. O que basicamente significa que ela precisa processar cada célula do tabuleiro, que tem tamanho $N \times N$, e isso é feito para cada uma das gerações que a gente quer simular. Então, se o tabuleiro for grande ou se o número de gerações for alto, o tempo de execução aumenta proporcionalmente. Como a função faz isso de forma recursiva, ela acaba repetindo esse processo para cada geração, por isso a multiplicação por g , o número de gerações.

5.5 Função `imprimeReticulado`

```

1 void imprimeReticulado(Automato *tab)
2 {
3     printf("\n");
4
5     for (int i = 0; i < tab->n; i++)
6     {
7         for (int j = 0; j < tab->n; j++)
8         {
9             int key = i * tab->n + j;
10            int valor = buscarHash(tab->tabela, key);
11            if (valor == -1)
12            {
13                printf("0 "); // Se nao encontrado na tabela, esta morto
14            }
15            else
16            {
17                printf("%d ", valor); // Imprime 1 se esta vivo
18            }

```

```

19     }
20     printf("\n");
21 }
22 }

```

A função `imprimeReticulado` é responsável por imprimir o estado atual do autômato na tela. Ela percorre toda a grade do autômato, verificando cada célula na tabela hash para determinar se está viva ou morta. Se a célula estiver viva, a função imprime 1; caso contrário, imprime 0. Essa abordagem permite visualizar claramente o estado do autômato no terminal, facilitando a análise da evolução das células ao longo das gerações. A complexidade dessa função é de $O(m^2)$, onde m é o tamanho da matriz $m \times m$. Essa complexidade se deve ao fato de que cada célula da matriz precisa ser visitada e impressa.

6 Arquivo de Implementação (tp.c)

O arquivo de implementação `tp.c` contém o código principal que utiliza as funções declaradas nos arquivos de cabeçalho `automato.h` e `double hash.h`, e implementadas em `automato.c` e `double hash.c`. Este arquivo orquestra a execução do programa, gerencia a entrada e saída de dados, e controla a evolução do autômato celular ao longo das gerações. A seguir, está descrito o funcionamento das principais seções e funções deste arquivo:

6.1 Inclusão de Cabeçalhos

```

1  include <stdio.h>
2  include <stdlib.h>
3  include "automato.h"

```

Inclui as bibliotecas necessárias para entrada e saída padrão e alocação de memória, além do arquivo de cabeçalho `automato.h`, que contém as declarações das funções e definições do `Automato`.

6.2 Função main

```

1  int main()
2  {
3      int n, geracoes;
4
5      // Le a dimensao do automato e o numero de geracoes.
6      scanf("%d %d", &n, &geracoes);
7
8      // Inicializa o automato
9      Automato *automato = alocarReticulado(n);
10
11     // Le o estado inicial do reticulado.
12     LeituraReticulado(automato);
13
14     // Evolui o automato para o numero especificado de geracoes.
15     automato = evoluirReticulado(automato, n, geracoes);
16
17     // Imprime o estado do reticulado apos as geracoes.
18     imprimeReticulado(automato);
19
20     // Desaloca a memoria usada pelo automato.
21     desalocarReticulado(&automato);
22
23     return 0;
24 }

```

A função `main` é o ponto de entrada do programa. Declara duas variáveis, `n` (dimensão do autômato) e `geracoes` (número de gerações a serem evoluídas). Utiliza `scanf` para ler esses valores da entrada padrão.

6.3 Chamadas das Funções

```
1  alocarReticulado(n);
2
3  LeituraReticulado(automato);
4
5  evoluirReticulado(automato, n, geracoes);
6
7  imprimeReticulado(automato);
8
9  desalocarReticulado(&automato);
```

- Inicialização: Chama a função `alocarReticulado` para alocar memória e inicializar o autômato com a dimensão especificada.
- Leitura do Reticulado: Chama a função `LeituraReticulado` para ler o estado inicial do tabuleiro a partir da entrada padrão.
- Evolução do Autômato: Chama a função `evoluirReticulado` para evoluir o autômato pelo número especificado de gerações.
- Impressão do Reticulado: Chama a função `imprimeReticulado` para imprimir o estado final do tabuleiro.
- Desalocação de Memória: Chama a função `desalocarReticulado` para liberar a memória alocada para o autômato, evitando vazamentos de memória.

6.4 Fim da Função main

```
1  return 0;
2  }
```

Retorna 0, indicando que o programa terminou com sucesso.

7 Impressões gerais

A implementação deste projeto foi uma experiência rica e desafiadora, especialmente no que se refere à utilização da Tabela Hash de Endereçamento Linear com endereçamento duplo. Essa abordagem se mostrou extremamente eficaz para gerenciar as inserções e buscas, minimizando colisões e garantindo a integridade dos dados armazenados. A escolha do endereçamento duplo foi uma estratégia acertada, pois permitiu uma distribuição mais uniforme das entradas, melhorando significativamente a performance em comparação com métodos mais simples.

Gerenciar a alocação e desalocação de memória na tabela hash foi uma tarefa que exigiu atenção aos detalhes, pois pequenos erros poderiam resultar em falhas de segmentação ou vazamentos de memória. A depuração e o tratamento de situações como a realocação de entradas foram desafiadores, mas essas dificuldades se transformaram em uma oportunidade valiosa para aprimorar meu entendimento sobre gerenciamento de memória em C. Cada erro encontrado serviu como um aprendizado sobre como projetar e implementar estruturas de dados que não apenas funcionam, mas também são eficientes e seguras.

A implementação das funções para manipular a tabela hash, incluindo a inserção, remoção e busca de elementos, me proporcionou um aprendizado profundo sobre como as tabelas hash operam internamente. A interação entre os métodos de hashing e as técnicas de resolução de colisões foi um aspecto fascinante do projeto, demonstrando como soluções algorítmicas inteligentes podem otimizar operações em grandes conjuntos de dados.

Além disso, a aplicação das regras do Jogo da Vida dentro dessa estrutura de dados ressaltou a importância de uma implementação robusta e eficiente. A capacidade de lidar com entradas dinâmicas e a complexidade da evolução do autômato foram facilitadas pela tabela hash, evidenciando sua relevância na resolução de problemas computacionais.

Em suma, esta experiência não apenas aprimorou minhas habilidades técnicas, mas também me proporcionou uma compreensão mais profunda do impacto das estruturas de dados na eficiência dos algoritmos. A Tabela Hash de Endereçamento Linear com endereçamento duplo se destacou como um componente crucial na arquitetura do projeto, reforçando a elegância e a eficácia das soluções algorítmicas bem fundamentadas.

8 Análise

Vamos analisar o tempo de execução do código e se teve algum vazamento de memória. Iremos também comparar os resultados deste trabalho com os 2 anteriores.

8.1 Tempo de Execução

Testamos o tempo de execução da entrada do teste 20.in. O resultado obtido foi:

```
1 real    0m1.105s
2 user    0m0.004s
3 sys     0m0.001s
```

8.2 Valgrind

Utilizamos o Valgrind para verificar se teve algum vazamento de memória, também com a entrada do teste 20.in. O resultado obtido foi:

```
1 ==23280== HEAP SUMMARY:
2 ==23280==      in use at exit: 0 bytes in 0 blocks
3 ==23280==    total heap usage: 10 allocs, 10 frees, 11,760 bytes allocated
4 ==23280==
5 ==23280== All heap blocks were freed -- no leaks are possible
6 ==23280==
7 ==23280== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

8.3 Comparação em relação aos trabalhos anteriores

Na comparação com os trabalhos anteriores, notamos diferenças significativas, especialmente em termos de otimização e uso de memória. No primeiro trabalho, desenvolvemos um autômato que armazenava os números e aplicava as regras do jogo da vida diretamente, resultando em um uso mais genérico de estruturas de dados, com maior dependência de alocações dinâmicas. Já no segundo, implementamos o sistema utilizando uma matriz esparsa, o que melhorou a eficiência em termos de armazenamento, já que grande parte da matriz do jogo da vida permanece vazia.

No terceiro trabalho, optamos por utilizar uma tabela hash, o que trouxe vantagens claras em termos de gestão de memória. Durante a análise com o Valgrind, percebemos uma redução considerável no número de alocações e desalocações em comparação com as versões anteriores. Isso acontece porque a tabela hash permite um acesso mais eficiente aos elementos, sem a necessidade de alocar grandes blocos de memória para armazenar posições ou células inativas do tabuleiro. Dessa forma, o terceiro trabalho se destaca por sua maior eficiência na utilização de memória e uma gestão mais enxuta dos recursos, com menos operações de alocação e liberação de memória.

9 Conclusão

Após a implementação e execução do terceiro trabalho sobre o Jogo da Vida, utilizando tabelas hash com endereçamento aberto e hashing duplo, podemos concluir que o desempenho e a eficiência do código foram satisfatórios. Utilizamos o comando time para medir o tempo de execução, e os resultados indicaram uma performance sólida, evidenciando a eficácia das funções implementadas e da abordagem escolhida.

Também empregamos a ferramenta Valgrind para verificar possíveis vazamentos de memória durante a execução do programa. Os resultados mostraram uma redução significativa no número de

alocações e desalocações, sem ocorrência de vazamentos, indicando que o uso da tabela hash proporcionou uma melhor gestão da memória em comparação com os trabalhos anteriores, onde foram utilizadas outras estruturas, como autômatos e matrizes esparsas.

O programa cumpriu todos os requisitos propostos, permitindo a inserção, busca e remoção de elementos de maneira eficiente. A implementação do endereçamento duplo na tabela hash demonstrou ser vantajosa, minimizando colisões e melhorando a performance das operações.

Em resumo, o projeto alcançou seus objetivos com um código eficiente e sem problemas de memória. A escolha da tabela hash para o terceiro trabalho foi acertada, resultando em uma implementação otimizada em relação ao gerenciamento de memória e desempenho, validando a eficácia das soluções e a estrutura de dados selecionada para este contexto.